LABSHEET - 9

HIMANI CHAUDHARY

BTECH(CSE) 3RD YEAR

CU240250967

SECTION - B

ROLL NO. - 24

Mastery in Data Analytics and Visualizations and Career Advancement (Technical)

```
# EXPERIMENT NO. 9
# DATA STORYTELLING AND BUSINESS INSIGHT GENERATION
# USING INTERACTIVE VISUALIZATIONS
# ============================================================

# ------------------------------------------------------------
# 1. IMPORT LIBRARIES
# ------------------------------------------------------------

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

print("=" * 70)
print("DATA STORYTELLING AND BUSINESS INSIGHT GENERATION")
print("=" * 70)
print("Himani Chaudhary")
print("CU240250967")
```

```
======================================================================
DATA STORYTELLING AND BUSINESS INSIGHT GENERATION
======================================================================
Himani Chaudhary
CU240250967
```

```
# 2. LOAD DATASET
# ------------------------------------------------------------

# IMPORTANT:
# Use r before the path so Windows backslashes are handled correctly.
# Also, .xls files must be read using read_excel(), NOT read_csv().

file_path = "/content/sample_-_superstore.xls"

try:
    df = pd.read_excel(file_path)

except FileNotFoundError:
    print("\nERROR: Dataset file not found!")
    print("\nPlease check the following:")
    print("1. The file exists at the specified location.")
    print("2. The filename is exactly:")
    print("   sample_-_superstore.xls")
    print("3. The folder path is correct.")
    raise SystemExit

except ImportError:
    print("\nERROR: Required Excel library is missing.")
    print("\nInstall xlrd using:")
    print("pip install xlrd")
    raise SystemExit

except Exception as e:
    print("\nERROR while reading the dataset:")
    print(e)
    raise SystemExit


print("\nDataset loaded successfully!")
print("Himani Chaudhary")
print("CU240250967")
```

```
Dataset loaded successfully!
Himani Chaudhary
CU240250967
```

```
# 3. DISPLAY BASIC INFORMATION
# ------------------------------------------------------------

print("\n" + "=" * 60)
print("DATASET INFORMATION")
print("=" * 60)

print("\nFirst 5 Records:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

print("\nColumn Names:")
print(df.columns.tolist())

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())
print("Himani Chaudhary")
print("CU240250967")
```

```
Ship Mode                object
Customer ID              object
Customer Name            object
Segment                  object
Country/Region           object
City                     object
State/Province           object
Postal Code              object
Region                   object
Product ID               object
Category                 object
Sub-Category             object
Product Name             object
Sales                   float64
Quantity                  int64
Discount                float64
Profit                  float64
dtype: object

Missing Values:
Row ID            0
Order ID          0
Order Date        0
Ship Date         0
Ship Mode         0
Customer ID       0
Customer Name     0
Segment           0
Country/Region    0
City              0
State/Province    0
Postal Code       0
Region            0
Product ID        0
Category          0
Sub-Category      0
Product Name      0
Sales             0
Quantity          0
Discount          0
Profit            0
dtype: int64
Himani Chaudhary
CU240250967
```

```
# 4. CLEAN COLUMN NAMES
# ------------------------------------------------------------

df.columns = df.columns.astype(str).str.strip()

print("\nCleaned Column Names:")
print(df.columns.tolist())
print("Himani Chaudhary")
print("CU240250967")
```

```
Cleaned Column Names:
['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Customer Name', 'Segment', 'Country/Region',
Himani Chaudhary
CU240250967
```

```
# 5. CONVERT DATE COLUMNS
# ------------------------------------------------------------

if "Order Date" in df.columns:

    df["Order Date"] = pd.to_datetime(
        df["Order Date"],
        errors="coerce"
    )

if "Ship Date" in df.columns:

    df["Ship Date"] = pd.to_datetime(
        df["Ship Date"],
        errors="coerce"
    )
    print("Himani Chaudhary")
    print("CU240250967")
```

```
Himani Chaudhary
CU240250967
```

```
# 6. CONVERT NUMERIC COLUMNS
# ------------------------------------------------------------

numeric_columns_to_convert = [
    "Sales",
    "Profit",
    "Quantity",
    "Discount",
    "Shipping Cost"
]

for column in numeric_columns_to_convert:

    if column in df.columns:

        df[column] = pd.to_numeric(
            df[column],
            errors="coerce"
        )
        print("Himani Chaudhary")
        print("CU240250967")
```

```
Himani Chaudhary
CU240250967
Himani Chaudhary
CU240250967
Himani Chaudhary
CU240250967
Himani Chaudhary
CU240250967
```

```
# 7. REMOVE DUPLICATE RECORDS
# ------------------------------------------------------------

duplicate_count = df.duplicated().sum()

print("\nNumber of Duplicate Records:", duplicate_count)

df = df.drop_duplicates()

print("Duplicates removed.")
print("New Dataset Shape:", df.shape)
print("Himani Chaudhary")
print("CU240250967")
```

```
Number of Duplicate Records: 0
Duplicates removed.
New Dataset Shape: (10194, 21)
Himani Chaudhary
CU240250967
```

```
# 8. BASIC STATISTICAL ANALYSIS
# ------------------------------------------------------------

print("\n" + "=" * 60)
print("STATISTICAL SUMMARY")
print("=" * 60)

print(df.describe(include="all"))
print("Himani Chaudhary")
print("CU240250967")
```

```
max     10194.000000             NaN            2026-12-30 00:00:00
std      2942.898656             NaN                            NaN

                           Ship Date       Ship Mode Customer ID  \
```

```
unique            800         3              2            542  ...
top     William Brown  Consumer  United States  New York City  ...
freq               41      5281           9994            915  ...
mean              NaN       NaN            NaN            NaN  ...
min               NaN       NaN            NaN            NaN  ...
25%               NaN       NaN            NaN            NaN  ...
50%               NaN       NaN            NaN            NaN  ...
75%               NaN       NaN            NaN            NaN  ...
max               NaN       NaN            NaN            NaN  ...
std               NaN       NaN            NaN            NaN  ...

       Postal Code  Region       Product ID         Category Sub-Category  \
count      10194.0   10194            10194            10194        10194
unique       654.0       4             1862                3           17
top        10035.0    West  FUR-FU-10004270  Office Supplies      Binders
freq         263.0    3253               20             6128         1548
mean           NaN     NaN              NaN              NaN          NaN
min            NaN     NaN              NaN              NaN          NaN
25%            NaN     NaN              NaN              NaN          NaN
50%            NaN     NaN              NaN              NaN          NaN
75%            NaN     NaN              NaN              NaN          NaN
max            NaN     NaN              NaN              NaN          NaN
std            NaN     NaN              NaN              NaN          NaN

       Product Name         Sales      Quantity      Discount        Profit
count         10194  10194.000000  10194.000000  10194.000000  10194.000000
unique         1849           NaN           NaN           NaN           NaN
top         Staples           NaN           NaN           NaN           NaN
freq              50           NaN           NaN           NaN           NaN
mean             NaN    228.225854      3.791838      0.155385     28.673417
min              NaN      0.444000      1.000000      0.000000  -6599.978000
25%              NaN     17.220000      2.000000      0.000000      1.760800
50%              NaN     53.910000      3.000000      0.200000      8.690000
75%              NaN    209.500000      5.000000      0.200000     29.297925
max              NaN  22638.480000     14.000000      0.800000   8399.976000
std              NaN    619.906839      2.228317      0.206249    232.465115

[11 rows x 21 columns]
Himani Chaudhary
CU240250967
```

```
# 9. KEY PERFORMANCE INDICATORS (KPIs)
# ------------------------------------------------------------

print("\n" + "=" * 60)
print("KEY PERFORMANCE INDICATORS")
print("=" * 60)

total_sales = 0
total_profit = 0
total_quantity = 0
total_customers = 0
total_orders = 0
profit_margin = 0


# Total Sales
if "Sales" in df.columns:

    total_sales = df["Sales"].sum()

    print(
        f"Total Sales       : ${total_sales:,.2f}"
    )


# Total Profit
if "Profit" in df.columns:

    total_profit = df["Profit"].sum()

    print(
        f"Total Profit      : ${total_profit:,.2f}"
    )


# Total Quantity
if "Quantity" in df.columns:

    total_quantity = df["Quantity"].sum()

    print(
        f"Total Quantity    : {total_quantity:,.0f}"
    )
```

```
# Total Customers
if "Customer ID" in df.columns:

    total_customers = df["Customer ID"].nunique()

    print(
        f"Total Customers   : {total_customers:,}"
    )


# Total Orders
if "Order ID" in df.columns:

    total_orders = df["Order ID"].nunique()

    print(
        f"Total Orders      : {total_orders:,}"
    )
    print("Himani Chaudhary")
    print("CU240250967")
```

```
============================================================
KEY PERFORMANCE INDICATORS
============================================================
Total Sales       : $2,326,534.35
Total Profit      : $292,296.81
Total Quantity    : 38,654
Total Customers   : 804
Total Orders      : 5,111
Himani Chaudhary
CU240250967
```

```
# 10. PROFIT MARGIN
# ------------------------------------------------------------

if (
    "Sales" in df.columns
    and "Profit" in df.columns
    and total_sales != 0
):

    profit_margin = (
        total_profit /
        total_sales
    ) * 100

    print(
        f"Profit Margin     : {profit_margin:.2f}%"
    )
    print("Himani Chaudhary")
    print("CU240250967")
```

```
Profit Margin     : 12.56%
Himani Chaudhary
CU240250967
```

```
# DATA STORY 1
# SALES BY CATEGORY
# ============================================================

category_sales = None

if (
    "Category" in df.columns
    and "Sales" in df.columns
):

    category_sales = (
        df.groupby("Category")["Sales"]
        .sum()
        .sort_values(ascending=False)
    )

    print("\n" + "=" * 60)
    print("SALES BY CATEGORY")
    print("=" * 60)

    print(category_sales)

    plt.figure(figsize=(10, 6))
```

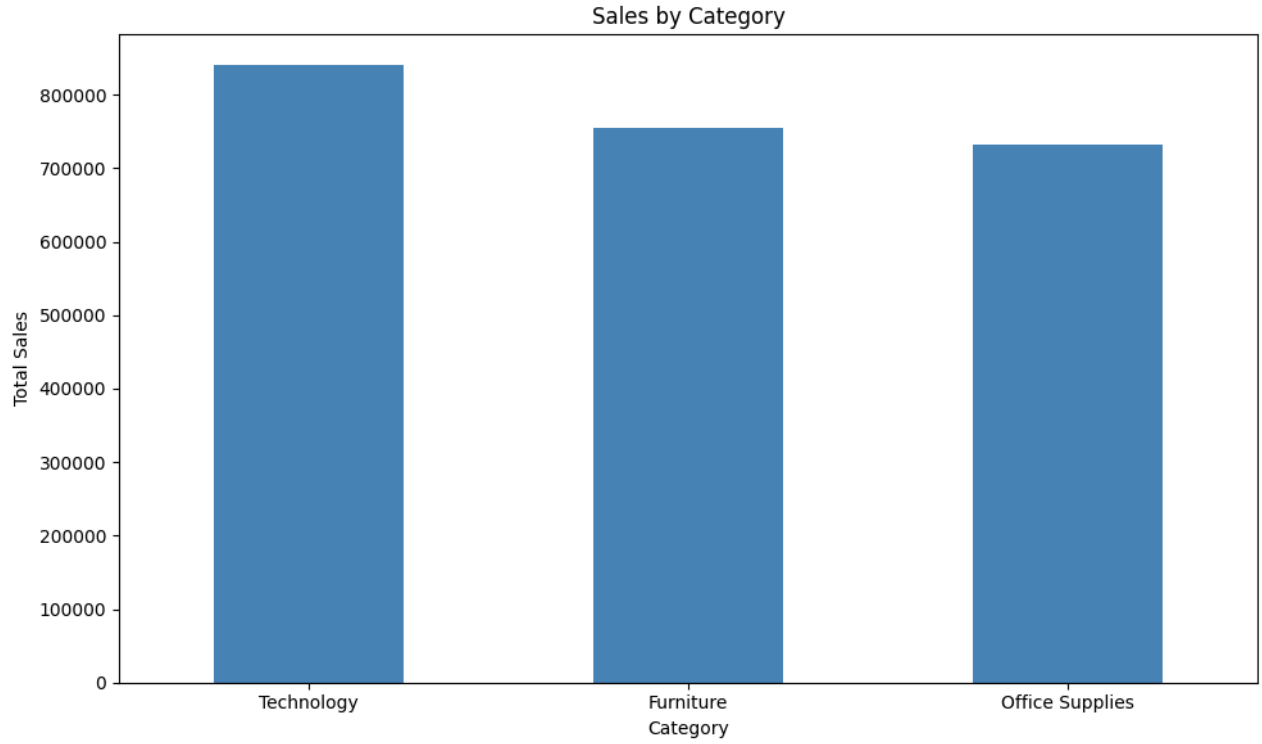
```
    category_sales.plot(
        kind="bar",
        color="steelblue"
    )

    plt.title("Sales by Category")
    plt.xlabel("Category")
    plt.ylabel("Total Sales")

    plt.xticks(rotation=0)

    plt.tight_layout()
    plt.show()
    print("Himani Chaudhary")
    print("CU240250967")
```

```
============================================================
SALES BY CATEGORY
============================================================
Category
Technology         839893.2790
Furniture          754747.7613
Office Supplies    731893.3140
Name: Sales, dtype: float64
```




```
Himani Chaudhary
CU240250967
```

```
# DATA STORY 2
# PROFIT BY CATEGORY
# ============================================================

category_profit = None

if (
    "Category" in df.columns
    and "Profit" in df.columns
):

    category_profit = (
        df.groupby("Category")["Profit"]
        .sum()
        .sort_values(ascending=False)
    )

    print("\n" + "=" * 60)
    print("PROFIT BY CATEGORY")
    print("=" * 60)
```

```
    print(category_profit)

    plt.figure(figsize=(10, 6))

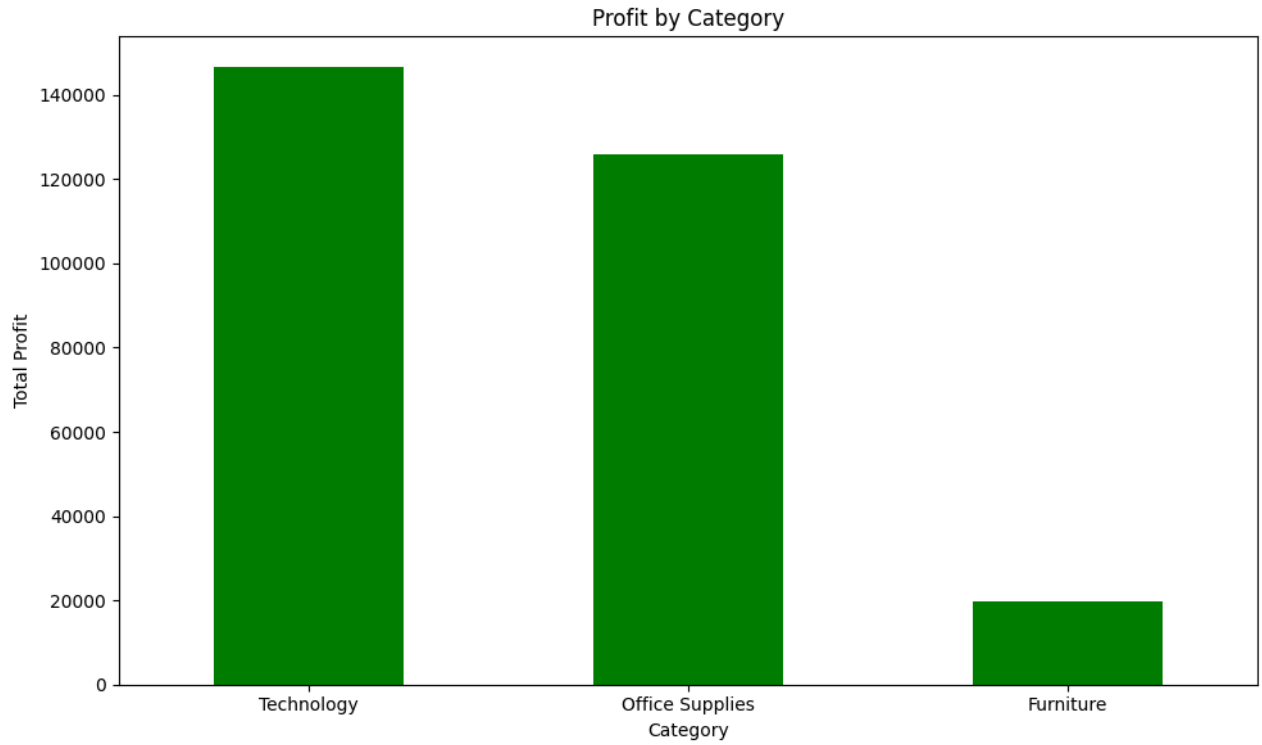
    category_profit.plot(
        kind="bar",
        color="green"
    )

    plt.title("Profit by Category")
    plt.xlabel("Category")
    plt.ylabel("Total Profit")

    plt.xticks(rotation=0)

    plt.tight_layout()
    plt.show()
    print("Himani Chaudhary")
    print("CU240250967")
```

```
============================================================
PROFIT BY CATEGORY
============================================================
Category
Technology         146543.3756
Office Supplies    126023.4434
Furniture           19729.9956
Name: Profit, dtype: float64
```




```
Himani Chaudhary
CU240250967
```

```
# DATA STORY 3
# SALES TREND OVER TIME
# ============================================================

monthly_sales = None

if (
    "Order Date" in df.columns
    and "Sales" in df.columns
):

    # Remove rows where Order Date is invalid
    trend_data = df.dropna(
        subset=["Order Date"]
    ).copy()

    if len(trend_data) > 0:
```

```
monthly_sales = (
    trend_data
    .set_index("Order Date")
    .resample("M")["Sales"]
    .sum()
)

print("\n" + "=" * 60)
print("MONTHLY SALES TREND")
print("=" * 60)

print(monthly_sales.head(12))

plt.figure(figsize=(14, 6))

plt.plot(
    monthly_sales.index,
    monthly_sales.values,
    marker="o",
    color="blue"
)

plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")

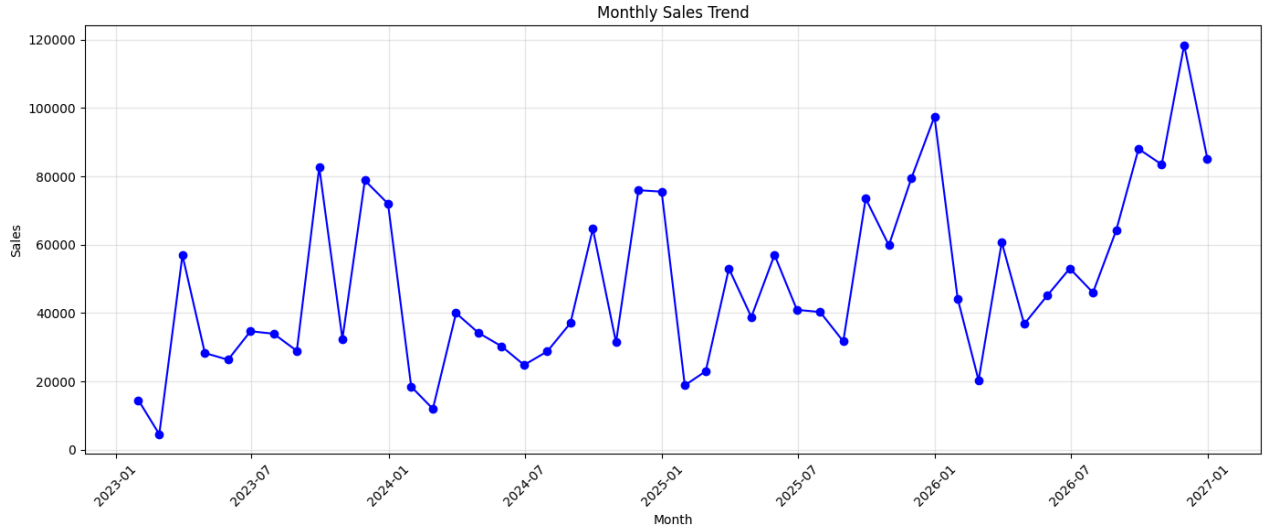
plt.xticks(rotation=45)

plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
print("Himani Chaudhary")
print("CU240250967")
```

```
/tmp/ipykernel_2779/1610452729.py:22: FutureWarning: 'M' is deprecated and will be removed in a future version, please use '
  .resample("M")["Sales"]

============================================================
MONTHLY SALES TREND
============================================================
Order Date
2023-01-31    14518.0550
2023-02-28     4519.8920
2023-03-31    56933.9090
2023-04-30    28295.3450
2023-05-31    26319.7670
2023-06-30    34669.4796
2023-07-31    33946.3930
2023-08-31    28918.3385
2023-09-30    82670.4288
2023-10-31    32413.3390
2023-11-30    78826.9567
2023-12-31    72008.3085
Freq: ME, Name: Sales, dtype: float64
```




```
Himani Chaudhary
CU240250967
```

```
# DATA STORY 4
# SALES BY REGION
# ============================================================

region_sales = None

if (
    "Region" in df.columns
    and "Sales" in df.columns
):

    region_sales = (
        df.groupby("Region")["Sales"]
        .sum()
        .sort_values(ascending=False)
    )

    print("\n" + "=" * 60)
    print("SALES BY REGION")
    print("=" * 60)

    print(region_sales)

    plt.figure(figsize=(10, 6))

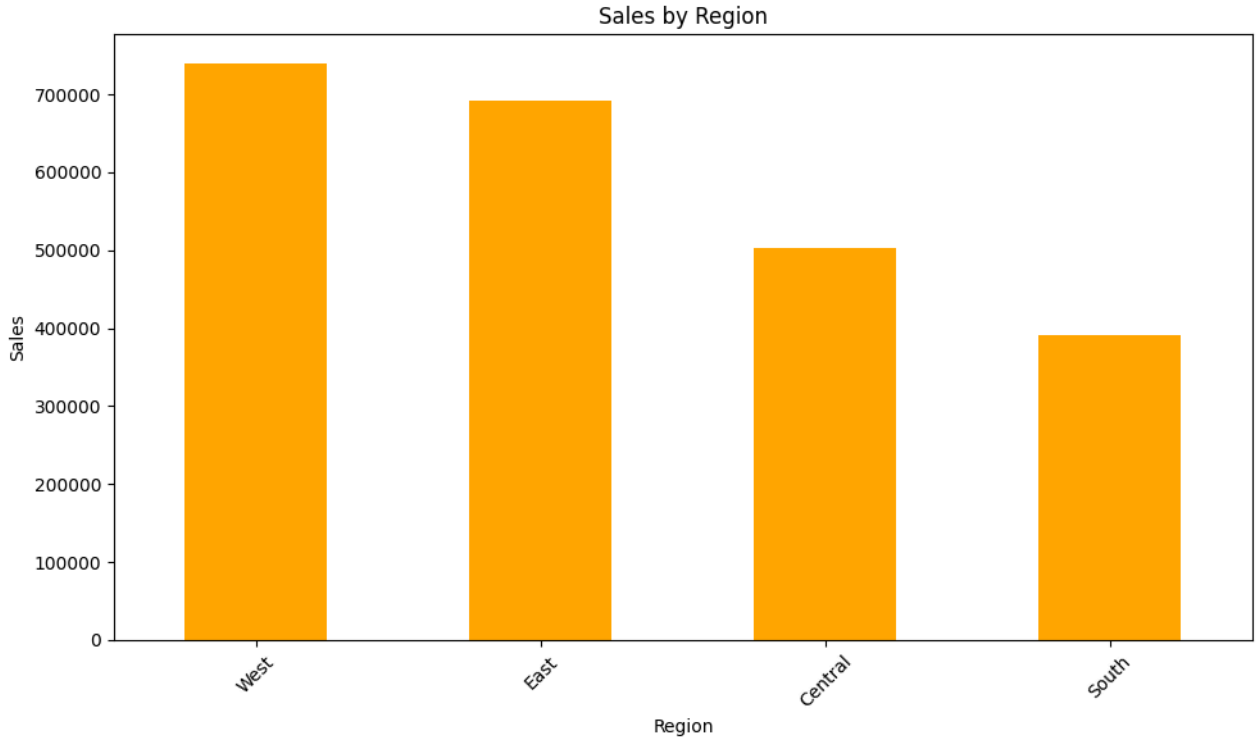
    region_sales.plot(
        kind="bar",
        color="orange"
    )

    plt.title("Sales by Region")
    plt.xlabel("Region")
    plt.ylabel("Sales")

    plt.xticks(rotation=45)

    plt.tight_layout()
    plt.show()
    print("Himani Chaudhary")
    print("CU240250967")
```

```
============================================================
SALES BY REGION
============================================================
Region
West       739813.6085
East       691828.1680
Central    503170.6728
South      391721.9050
Name: Sales, dtype: float64
```




```
Himani Chaudhary
CU240250967
```

```
# DATA STORY 5
# PROFIT BY REGION
# ============================================================

region_profit = None

if (
    "Region" in df.columns
    and "Profit" in df.columns
):

    region_profit = (
        df.groupby("Region")["Profit"]
        .sum()
        .sort_values(ascending=False)
    )

    print("\n" + "=" * 60)
    print("PROFIT BY REGION")
    print("=" * 60)

    print(region_profit)

    plt.figure(figsize=(10, 6))

    region_profit.plot(
        kind="bar",
        color="purple"
    )

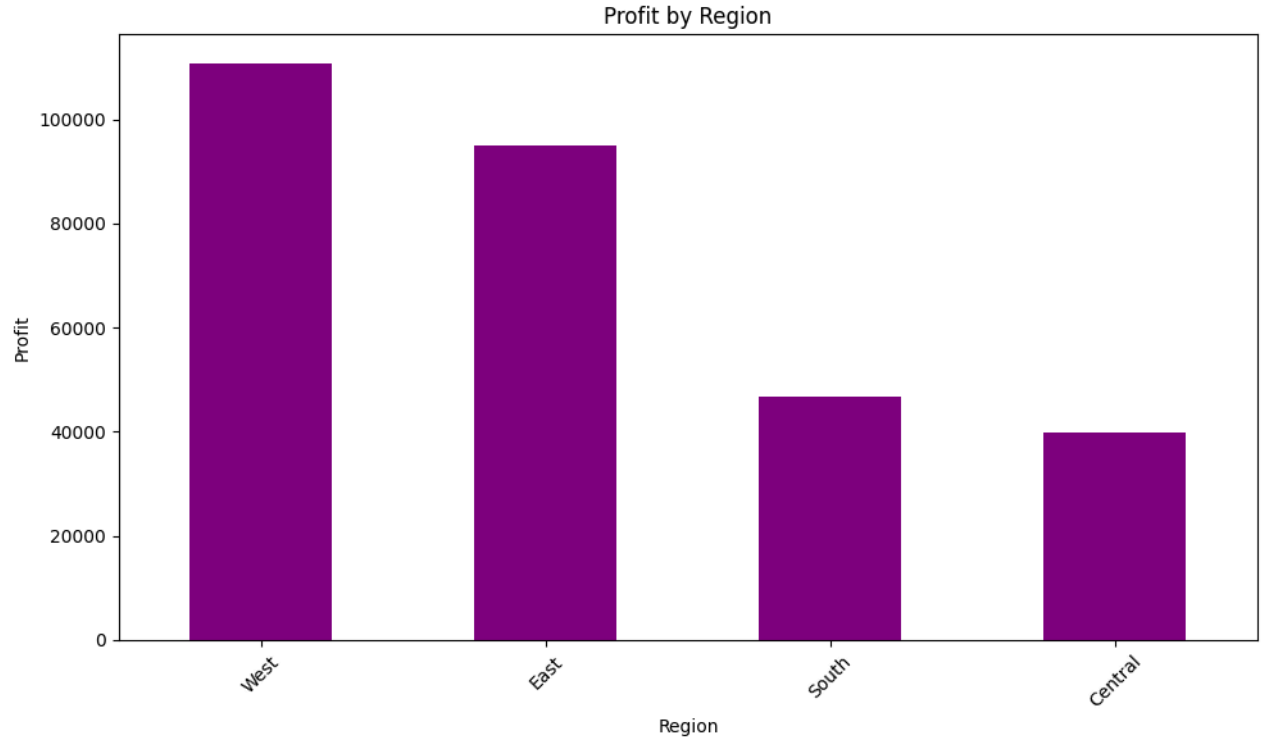
    plt.title("Profit by Region")
    plt.xlabel("Region")
    plt.ylabel("Profit")

    plt.xticks(rotation=45)

    plt.tight_layout()
    plt.show()
```

```
    print("Himani Chaudhary")
    print("CU240250967")
```

```
============================================================
PROFIT BY REGION
============================================================
Region
West       110798.8170
East        94883.2603
South       46749.4303
Central     39865.3070
Name: Profit, dtype: float64
```




```
Himani Chaudhary
CU240250967
```

```
# DATA STORY 6
# SALES BY SEGMENT
# ============================================================

segment_sales = None

if (
    "Segment" in df.columns
    and "Sales" in df.columns
):

    segment_sales = (
        df.groupby("Segment")["Sales"]
        .sum()
        .sort_values(ascending=False)
    )

    print("\n" + "=" * 60)
    print("SALES BY CUSTOMER SEGMENT")
    print("=" * 60)

    print(segment_sales)

    plt.figure(figsize=(10, 6))

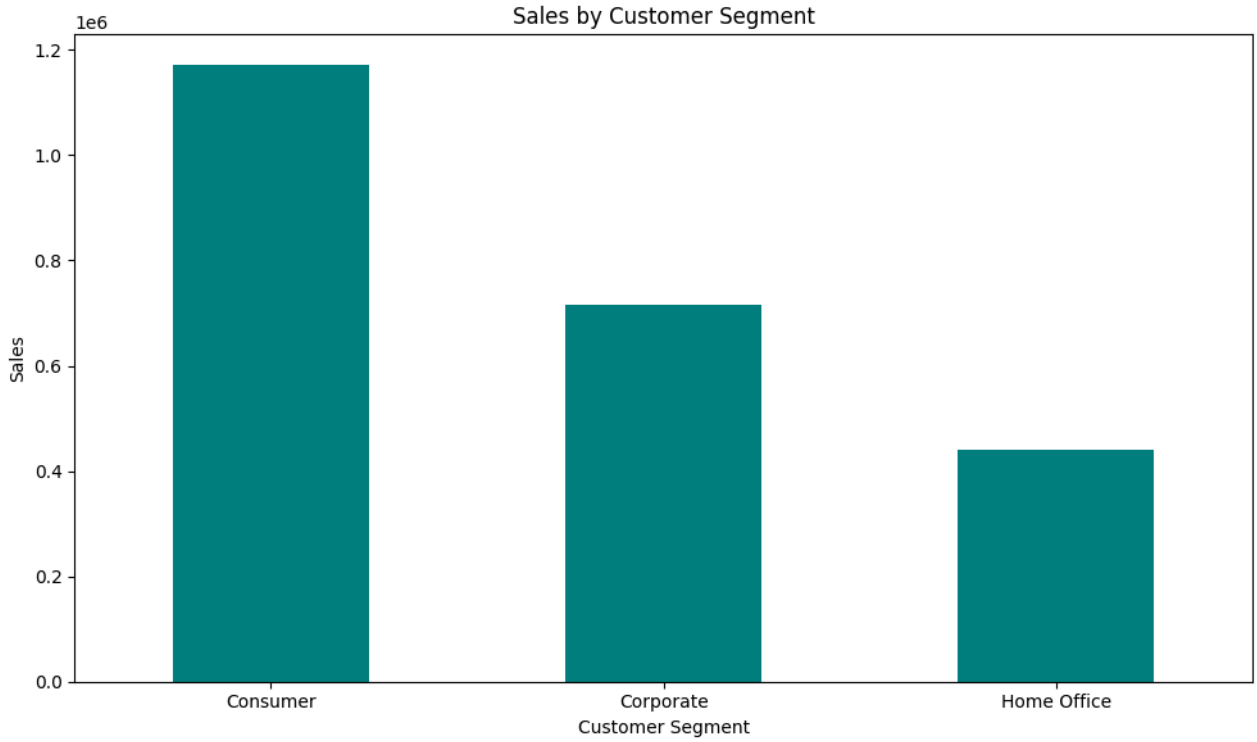
    segment_sales.plot(
        kind="bar",
        color="teal"
    )

    plt.title("Sales by Customer Segment")
    plt.xlabel("Customer Segment")
    plt.ylabel("Sales")

    plt.xticks(rotation=0)
```

```
    plt.tight_layout()
    plt.show()
    print("Himani Chaudhary")
    print("CU240250967")
```

```
============================================================
SALES BY CUSTOMER SEGMENT
============================================================
Segment
Consumer       1.170660e+06
Corporate      7.158061e+05
Home Office    4.400684e+05
Name: Sales, dtype: float64
```




```
Himani Chaudhary
CU240250967
```

```
# DATA STORY 7
# TOP 10 SUB-CATEGORIES
# ============================================================

top_subcategories = None

if (
    "Sub-Category" in df.columns
    and "Sales" in df.columns
):

    top_subcategories = (
        df.groupby("Sub-Category")["Sales"]
        .sum()
        .sort_values(ascending=False)
        .head(10)
    )

    print("\n" + "=" * 60)
    print("TOP 10 SUB-CATEGORIES BY SALES")
    print("=" * 60)

    print(top_subcategories)

    plt.figure(figsize=(12, 6))

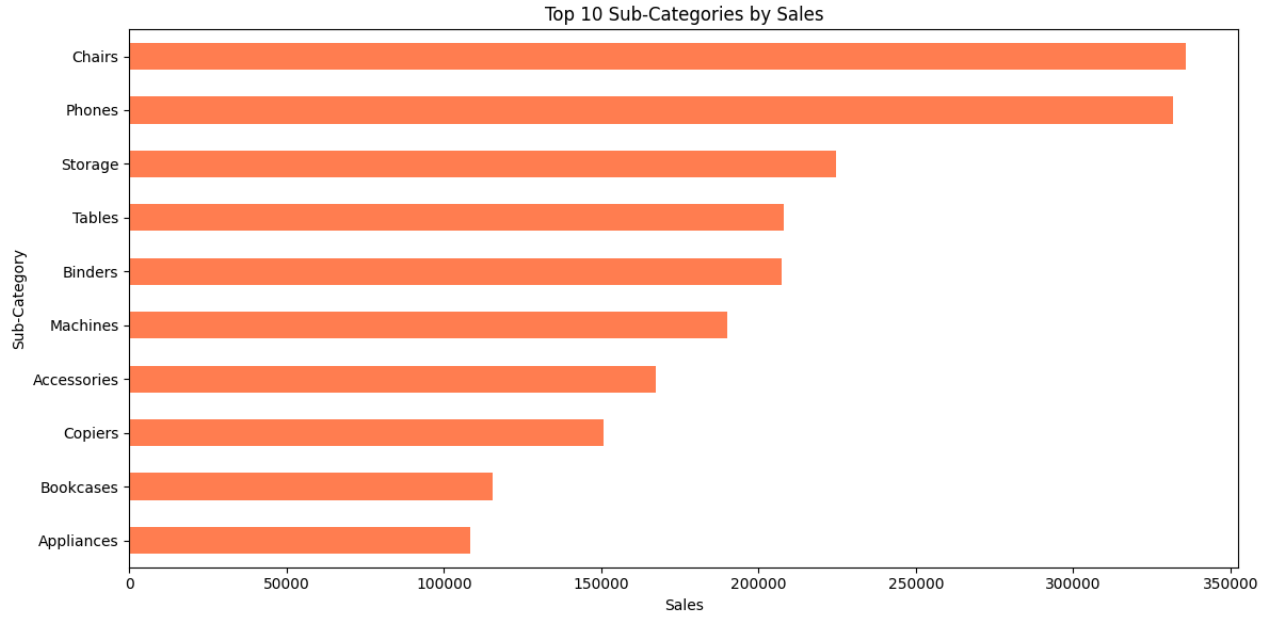
    top_subcategories.sort_values().plot(
        kind="barh",
        color="coral"
    )

    plt.title("Top 10 Sub-Categories by Sales")
```

```
    plt.xlabel("Sales")
    plt.ylabel("Sub-Category")

    plt.tight_layout()
    plt.show()
    print("Himani Chaudhary")
    print("CU240250967")
```

```
============================================================
TOP 10 SUB-CATEGORIES BY SALES
============================================================
Sub-Category
Chairs         335768.2490
Phones         331842.6400
Storage        224644.5540
Tables         208020.1820
Binders        207354.8810
Machines       189925.0310
Accessories    167380.3180
Copiers        150745.2900
Bookcases      115361.2043
Appliances     108213.1850
Name: Sales, dtype: float64
```




```
Himani Chaudhary
CU240250967
```

```
# DATA STORY 8
# TOP 10 PRODUCTS
# ============================================================

top_products = None

if (
    "Product Name" in df.columns
    and "Sales" in df.columns
):

    top_products = (
        df.groupby("Product Name")["Sales"]
        .sum()
        .sort_values(ascending=False)
        .head(10)
    )

    print("\n" + "=" * 60)
    print("TOP 10 PRODUCTS BY SALES")
    print("=" * 60)

    print(top_products)
    print("Himani Chaudhary")
    print("CU240250967")
```

```
============================================================
```

```
TOP 10 PRODUCTS BY SALES
============================================================
Product Name
Canon imageCLASS 2200 Advanced Copier                                    61599.824
Fellowes PB500 Electric Punch Plastic Comb Binding Machine with Manual Bind    27453.384
Cisco TelePresence System EX90 Videoconferencing Unit                    22638.480
HON 5400 Series Task Chairs for Big and Tall                             21870.576
GBC DocuBind TL300 Electric Binding System                               19823.479
GBC Ibimaster 500 Manual ProClick Binding System                         19024.500
Hewlett Packard LaserJet 3310 Copier                                     18839.686
HP Designjet T520 Inkjet Large Format Printer - 24" Color                18374.895
GBC DocuBind P400 Electric Binding System                                17965.068
High Speed Automatic Electric Letter Opener                              17030.312
Name: Sales, dtype: float64
Himani Chaudhary
CU240250967
```

```
# DATA STORY 9
# CORRELATION HEATMAP
# ============================================================

numeric_columns = df.select_dtypes(
    include=np.number
).columns

if len(numeric_columns) > 1:

    correlation = df[numeric_columns].corr()

    print("\n" + "=" * 60)
    print("CORRELATION MATRIX")
    print("=" * 60)

    print(correlation)

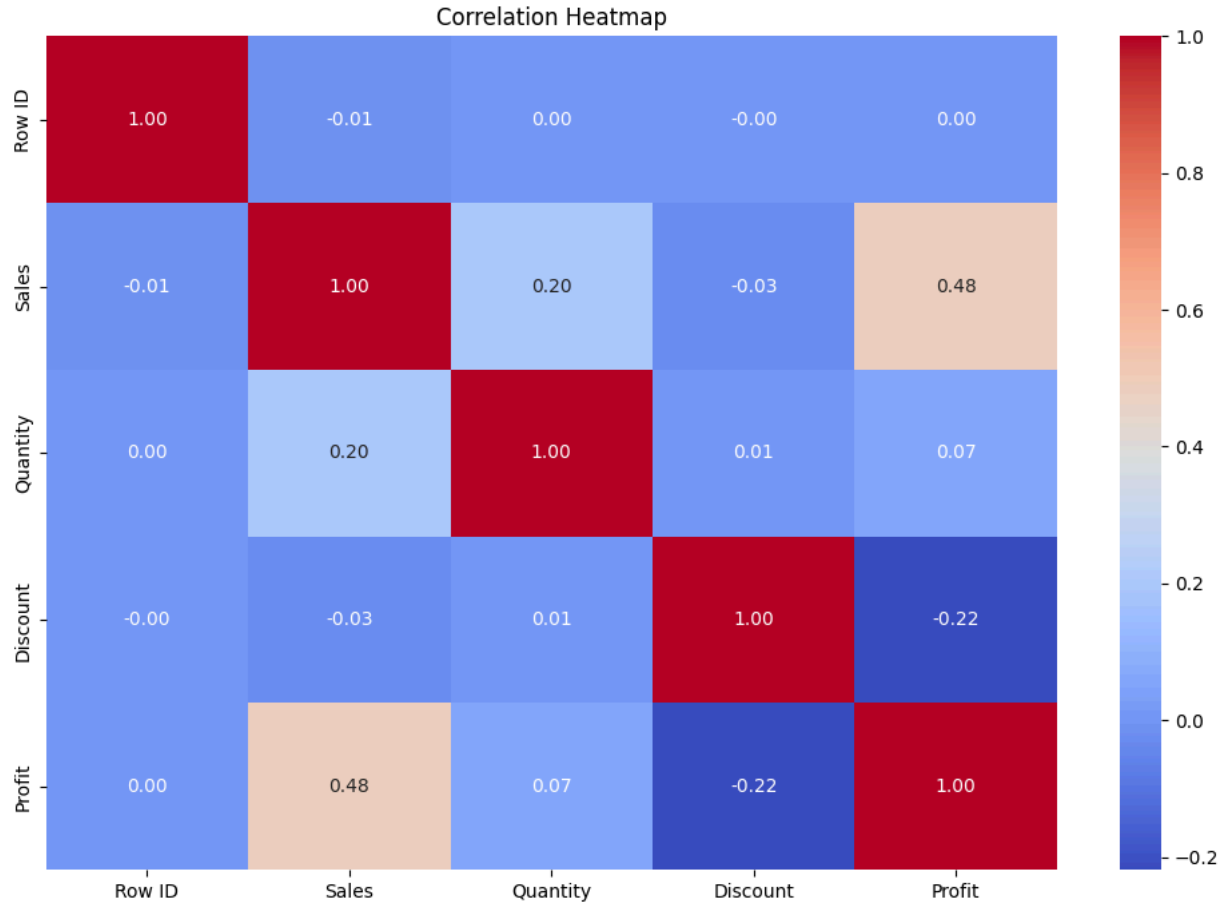
    plt.figure(figsize=(10, 7))

    sns.heatmap(
        correlation,
        annot=True,
        cmap="coolwarm",
        fmt=".2f"
    )

    plt.title("Correlation Heatmap")

    plt.tight_layout()
    plt.show()
```

```
============================================================
CORRELATION MATRIX
============================================================
             Row ID     Sales  Quantity  Discount    Profit
Row ID    1.000000 -0.008958  0.000502 -0.001474  0.003022
Sales    -0.008958  1.000000  0.198457 -0.027756  0.481460
Quantity  0.000502  0.198457  1.000000  0.007475  0.066145
Discount -0.001474 -0.027756  0.007475  1.000000 -0.218882
Profit    0.003022  0.481460  0.066145 -0.218882  1.000000
```




```
# DATA STORY 10
# SALES VS PROFIT
# ============================================================

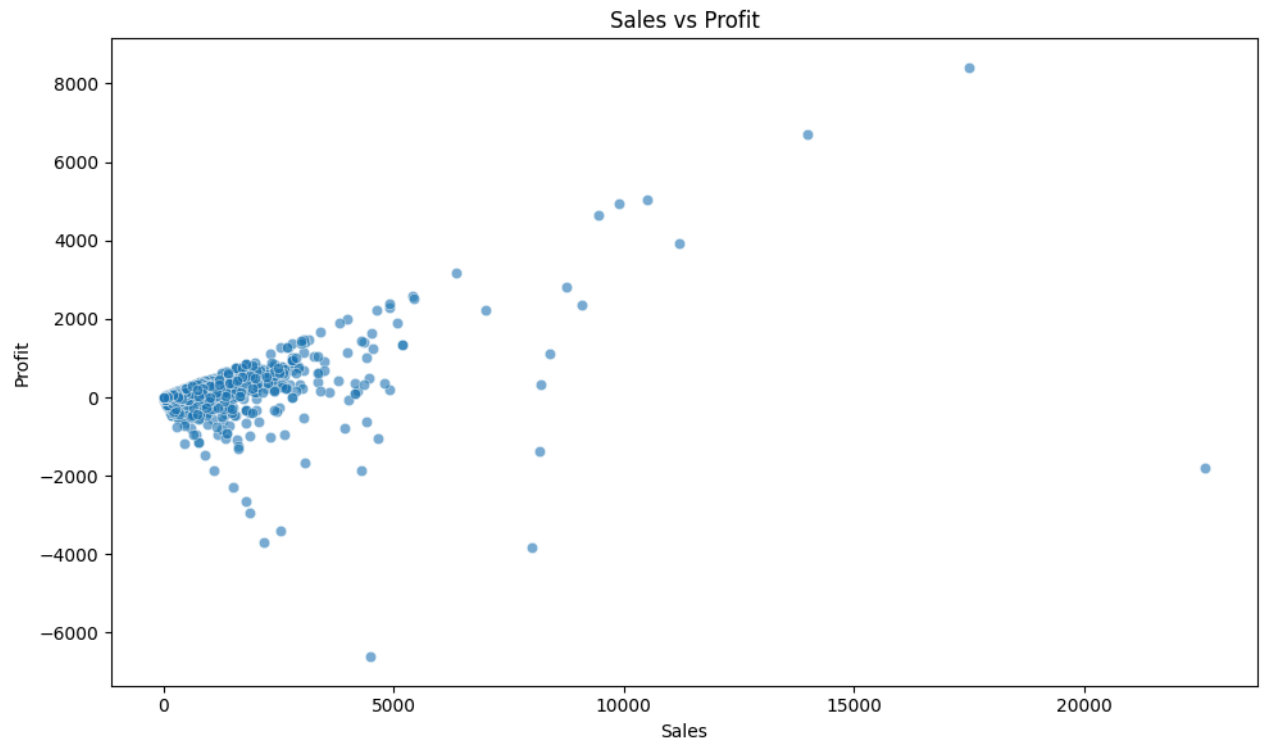
if (
    "Sales" in df.columns
    and "Profit" in df.columns
):

    plt.figure(figsize=(10, 6))

    sns.scatterplot(
        data=df,
        x="Sales",
        y="Profit",
        alpha=0.6
    )

    plt.title("Sales vs Profit")
    plt.xlabel("Sales")
    plt.ylabel("Profit")

    plt.tight_layout()
    plt.show()
    print("Himani Chaudhary")
    print("CU240250967")
```



Himani Chaudhary
CU240250967

```
# INTERACTIVE VISUALIZATION USING PLOTLY
# ============================================================


# ------------------------------------------------------------
# Interactive Sales by Category
# ------------------------------------------------------------

if category_sales is not None:

    category_df = (
        category_sales
        .reset_index()
    )

    category_df.columns = [
        "Category",
        "Sales"
    ]

    fig1 = px.bar(
        category_df,
        x="Category",
        y="Sales",
        title="Interactive Sales by Category",
        text_auto=".2s",
        color="Category"
    )

    fig1.update_layout(
        xaxis_title="Category",
        yaxis_title="Sales"
    )

    fig1.show()


# ------------------------------------------------------------
# Interactive Sales by Region
# ------------------------------------------------------------

if region_sales is not None:

    region_df = (
        region_sales
        .reset_index()
    )
```

```
    region_df.columns = [
        "Region",
        "Sales"
    ]

    fig2 = px.bar(
        region_df,
        x="Region",
        y="Sales",
        title="Interactive Sales by Region",
        text_auto=".2s",
        color="Region"
    )

    fig2.update_layout(
        xaxis_title="Region",
        yaxis_title="Sales"
    )

    fig2.show()


# ------------------------------------------------------------
# Interactive Sales Trend
# ------------------------------------------------------------

if monthly_sales is not None:

    trend_df = (
        monthly_sales
        .reset_index()
    )

    trend_df.columns = [
        "Date",
        "Sales"
    ]

    fig3 = px.line(
        trend_df,
        x="Date",
        y="Sales",
        title="Interactive Monthly Sales Trend",
        markers=True
    )

    fig3.update_layout(
        xaxis_title="Month",
        yaxis_title="Sales"
    )

    fig3.show()


# ============================================================
# FIND BEST AND WORST PERFORMING CATEGORIES
# ============================================================

best_category = None
worst_category = None

if category_sales is not None:

    best_category = category_sales.idxmax()
    worst_category = category_sales.idxmin()

    print("\n" + "=" * 60)
    print("CATEGORY INSIGHTS")
    print("=" * 60)

    print(
        "Highest Sales Category :",
        best_category
    )

    print(
        "Lowest Sales Category  :",
        worst_category
    )
```

```
# ============================================================
# FIND BEST AND WORST REGIONS
# ============================================================

best_region = None
worst_region = None

if region_sales is not None:

    best_region = region_sales.idxmax()
    worst_region = region_sales.idxmin()

    print("\n" + "=" * 60)
    print("REGION INSIGHTS")
    print("=" * 60)

    print(
        "Highest Sales Region :",
        best_region
    )

    print(
        "Lowest Sales Region  :",
        worst_region
    )


# ============================================================
# FIND MOST PROFITABLE CATEGORY
# ============================================================

best_profit_category = None
worst_profit_category = None

if category_profit is not None:

    best_profit_category = category_profit.idxmax()
    worst_profit_category = category_profit.idxmin()

    print("\n" + "=" * 60)
    print("PROFITABILITY INSIGHTS")
    print("=" * 60)

    print(
        "Most Profitable Category :",
        best_profit_category
    )

    print(
        "Least Profitable Category:",
        worst_profit_category
    )


# ============================================================
# BUSINESS RECOMMENDATIONS
# ============================================================

print("\n" + "=" * 60)
print("BUSINESS RECOMMENDATIONS")
print("=" * 60)

if best_category is not None:

    print(
        f"1. Focus marketing efforts on the "
        f"{best_category} category."
    )

if best_region is not None:

    print(
        f"2. Strengthen sales strategies in the "
        f"{best_region} region."
    )

if worst_region is not None:

    print(
        f"3. Analyze the reasons for lower sales in the "
        f"{worst_region} region."
    )
```

```
if best_profit_category is not None:

    print(
        f"4. Increase attention to the "
        f"{best_profit_category} category because it "
        f"generates the highest total profit."
    )

print(
    "5. Monitor monthly sales trends to identify "
    "seasonal opportunities."
)

print(
    "6. Use interactive dashboards to continuously "
    "monitor important KPIs."
)


# ============================================================
# FINAL DATA STORY
# ============================================================

print("\n" + "=" * 60)
print("FINAL DATA STORY")
print("=" * 60)

print(
    "\nThe analysis of the Superstore dataset "
    "provides useful insights into sales and profitability."
)

if best_category is not None:

    print(
        f"- {best_category} is the highest-selling category."
    )

if best_region is not None:

    print(
        f"- {best_region} is the highest-selling region."
    )

if best_profit_category is not None:

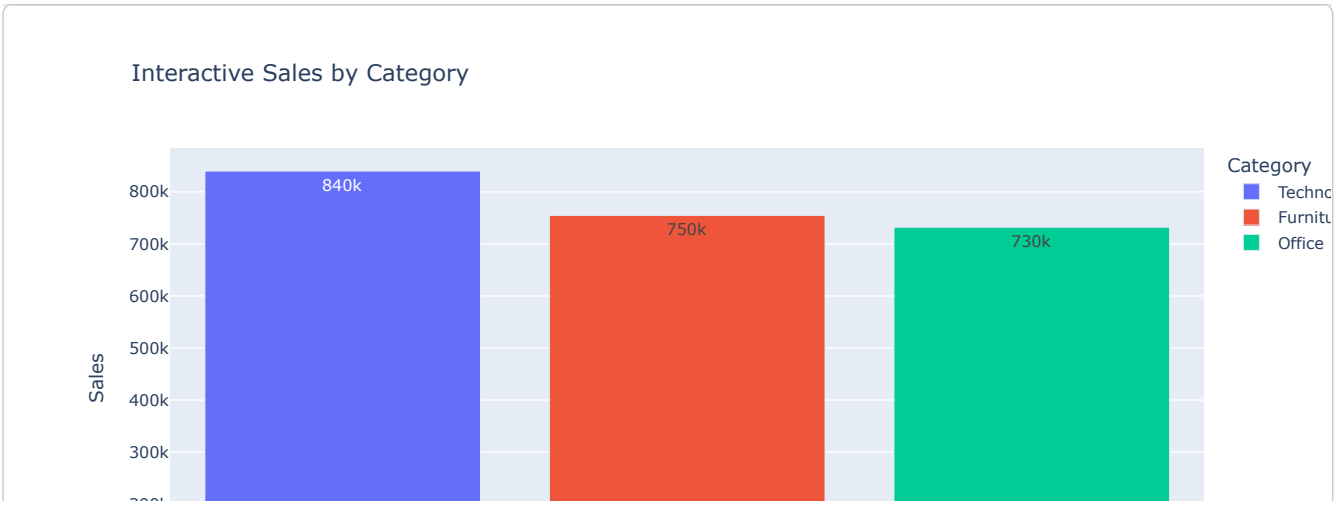
    print(
        f"- {best_profit_category} is the most profitable category."
    )

if (
    "Sales" in df.columns
    and "Profit" in df.columns
):

    print(
        f"- Overall sales are ${total_sales:,.2f} "
        f"with total profit of ${total_profit:,.2f}."
    )

    print(
        f"- Overall profit margin is {profit_margin:.2f}%."
    )

print(
    "\nThese insights can help management make "
    "data-driven business decisions."
)
print("Himani Chaudhary")
print("CU240250967")
```


Interactive Sales by Category
Sales
800k
700k
600k
500k
400k
300k
840k
750k
730k
Category
Techno
Furnitu
Office